NESTED QUERIES

One of the most powerful features of SQL is nested queries. A **nested query** is a query that has another query embedded within it; the embedded query is called a **subquery**. When writing a query, we sometimes need to express a condition that refers to a table that must itself be computed. The query used to compute this subsidiary table is a subquery and appears as part of the main query. A subquery typically appears within the `WHERE` clause of a query. Subqueries can sometimes appear in the `FROM` clause or the `HAVING` clause.

Nested Subqueries

- SQL provides a mechanism for the nesting of subqueries. A **subquery** is a **select-from-where** expression that is nested within another query.
- The nesting can be done in the following SQL query

  select $A_1, A_2, ..., A_n$
  from $r_1, r_2, ..., r_m$
  where P

  as follows:

  - **From clause:** r_i can be replaced by any valid subquery
  - **Where clause:** P can be replaced with an expression of the form:

    B <operation> (subquery)

    B is an attribute and <operation> to be defined later.
  - **Select clause:**

    A_i can be replaced be a subquery that generates a single value.

Nested Subqueries: Set Membership (**IN and NOT IN**)

How many A values also exist in B

R

A
2
3
4
5

S

B
4
5
6
9

O/P

A
4
5

Select A from R where R.A IN ..... (Select B from S)

$2 = \{4, 5, 6, 9\}$ ✗

Set-Comparison Operators:

ANY or SOME

ALL

SQL also supports op ANY and op ALL, where op is one of the arithmetic comparison operators $\{<, <=, =, <>, >=, >\}$. (SOME is also available, but it is just a synonym for ANY.)

$2 >= \{4, 5, 6, 9\}$ ✗

$2 >= \text{ANY}\{4, 5, 6, 9\}$ ✓

R

A
2
3
4
5

S

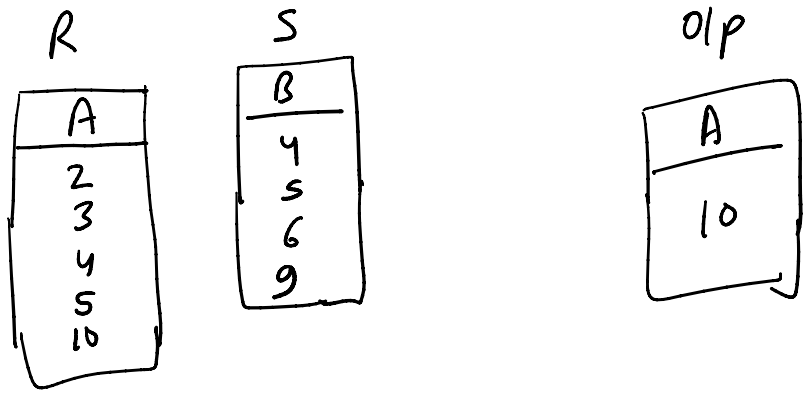
B
4
5
6
9

O/P

A
4
5

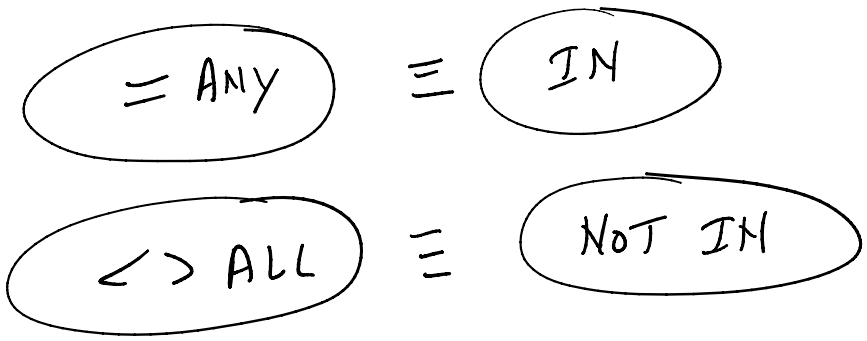
Select A from R where R.A >= ANY (Select B from S)

{4, 5, 6, 9}



Select A from R where R.A >= ALL (Select B from S)

Some values of A will be greater than equal to all values of B



Note that `IN` and `NOT IN` are equivalent to = `ANY` and <> `ALL`, respectively.

section(*course_id*, *sec_id*, *semester*, *year*, *building*, *room_number*, *time_slot_id*)

- Find all the courses taught in both the Fall 2009 and Spring 2010 semesters.

```
select distinct course_id
from section
where semester = 'Fall' and year= 2009 and
        course_id in (select course_id
                      from section
                      where semester = 'Spring' and year= 2010);
```

section(*course_id*, *sec_id*, *semester*, *year*, *building*, *room_number*, *time_slot_id*)

- Find all the courses taught in the Fall 2009 semester but not in the Spring 2010 semester.

```
select distinct course_id
from section
where semester = 'Fall' and year= 2009 and
course_id not in (select course_id
                  from section
                  where semester = 'Spring' and year= 2010);
```

instructor(*ID*, *name*, *dept_name*, *salary*)

takes(*ID*, *course_id*, *sec_id*, *semester*, *year*, *grade*)

- Find the names of instructors whose names are neither "Mozart" nor "Einstein".

```
select distinct name
from instructor
where name not in ('Mozart', 'Einstein');
```

- Find the total number of (distinct) students who have taken course sections taught by the instructor with ID 10101.

```
select count (distinct ID)
from takes
where (course_id, sec_id, semester, year) in (select course_id, sec_id, semester, year
                                              from teaches
                                              where teaches.ID= 10101);
```

instructor(*ID*, *name*, *dept_name*, *salary*)

- Find the names of all instructors whose salary is greater than at least one instructor in the Biology department.

```
select distinct T.name
from instructor as T, instructor as S
where T.salary > S.salary and S.dept_name = 'Biology';
```

```
select name
from instructor
where salary > some (select salary
                     from instructor
                     where dept_name = 'Biology');
```

Definition of "some" Clause

- F <comp> **some** $r \Leftrightarrow \exists\, t \in r$ such that (F <comp> t)
  Where <comp> can be: $<,\ \leq,\ >,\ =,\ \neq$

(5 < **some** | 0 / 5 / 6 |) = true (read: 5 < some tuple in the relation)

(5 < **some** | 0 / 5 |) = false

(5 = **some** | 0 / 5 |) = true

(5 ≠ **some** | 0 / 5 |) = true (since 0 ≠ 5)

(= **some**) ≡ **in**
However, (≠ **some**) ≠ **not in**

instructor(<u>*ID*</u>, *name*, *dept_name*, *salary*)

- Find the names of all instructors who have a salary value greater than that of each instructor in the Biology department.

```
select name
from instructor
where salary > all (select salary
                    from instructor
                    where dept_name = 'Biology');
```

Definition of “all” Clause

- F <comp> **all** $r \Leftrightarrow \forall\ t \in r$ (F <comp> *t)*

(5 < **all** | 0 / 5 / 6 |) = false

(5 < **all** | 6 / 10 |) = true

(5 = **all** | 4 / 5 |) = false

(5 ≠ **all** | 4 / 6 |) = true (since 5 ≠ 4 and 5 ≠ 6)

(≠ **all**) ≡ **not in**
However, (= **all**) ≢ **in**

Correlated Nested Queries

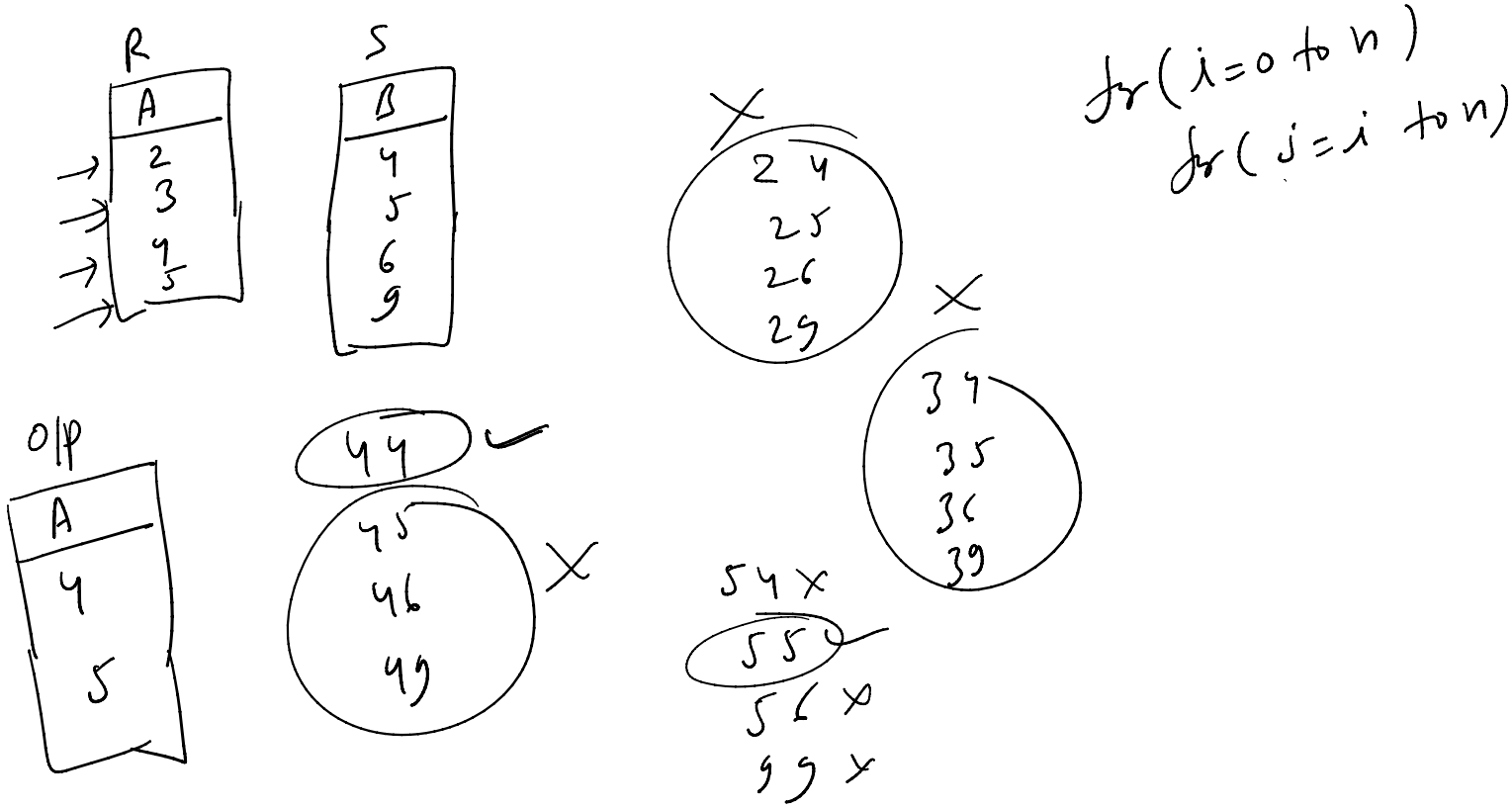
In the nested queries that we have seen thus far, the inner subquery has been completely independent of the outer query. In general the inner subquery could depend on the row that is currently being examined in the outer query.

EXISTS and NOT EXISTS

The EXISTS operator is a set comparison operator. It allows us to test whether a set is nonempty.

Select A from R where R.A …… (Select * from S where R.A = S.B)

Select A from R where EXISTS (Select * from S where R.A = S.B)



Test for Empty Relations

- The **exists** construct returns the value **true** if the argument subquery is nonempty.
- **exists** $r \Leftrightarrow r \neq \emptyset$
- **not exists** $r \Leftrightarrow r = \emptyset$

section(*course_id*, *sec_id*, *semester*, *year*, *building*, *room_number*, *time_slot_id*)

- Find all courses taught in both the Fall 2009 semester and in the Spring 2010 semester.

```
select course_id
from section as S
where semester = 'Fall' and year= 2009 and
        exists (select *
                from section as T
                where semester = 'Spring' and year= 2010 and
                        S.course_id= T.course_id);
```

Correlated subquery

Test for Absence of Duplicate Tuples

- The **unique** construct tests whether a subquery has any duplicate tuples in its result.
- The **unique** construct evaluates to "true" if a given subquery contains no duplicates .

course(*course_id*, *title*, *dept_name*, *credits*)

section(*course_id*, *sec_id*, *semester*, *year*, *building*, *room_number*, *time_slot_id*)

- Find all courses that were offered exactly once in 2009.

```
select T.course_id
from course as T
where unique (select R.course_id
              from section as R
              where T.course_id= R.course_id and
                    R.year = 2009);
```

Modification of the Database

- Deletion of tuples from a given relation.
- Insertion of new tuples into a given relation
- Updating of values in some tuples in a given relation

Deletion

- Delete all instructors

  delete from *instructor*

- Delete all instructors from the Finance department

  delete from *instructor*
  where *dept_name*= 'Finance';

Insertion

- Add a new tuple to *course*

  insert into *course*
  values ('CS-437', 'Database Systems', 'Comp. Sci.', 4);

- or equivalently

  insert into *course* (*course_id*, *title*, *dept_name*, *credits*)
  values ('CS-437', 'Database Systems', 'Comp. Sci.', 4);

- Add a new tuple to *student* with *tot_creds* set to null

  insert into *student*
  values ('3003', 'Green', 'Finance', *null*);

Updates

- Give a 5% salary raise to all instructors

```
update instructor
   set salary = salary * 1.05
```

- Give a 5% salary raise to those instructors who earn less than 70000

```
update instructor
   set salary = salary * 1.05
  where salary < 70000;
```